Department of Information Engineering Technology
The Superior University, Lahore

Mobile Application Development Lab

Experiment No.4

Flutter: Introduction to widgets

Prepared for

Mam Esha Nawaz

By:

Name: Ahmad Hassan

ID: SU91-BIETM-F23-027

Section: A

Semester: 6th

Total Marks: _____

Obtained Marks: _____

Signature: _____

Date: _____

Experiment No.4

Flutter: Introduction to widgets

(Rubrics)

Name: Ahmad Hassan **Roll No:** SU91-BIETM-F23-027

A. PSYCHOMOTOR

Sr. No.	Criteria	Allocated Marks	Unacceptable 0%	Poor 25%	Fair 50%	Good 75%	Excellent 100%	Total Obtained
1	Follow Procedures	1	0	0.25	0.5	0.75	1	
2	Software and Simulations	2	0	0.5	1	1.5	2	
3	Accuracy in Output Results	3	0	0.75	1.5	2.25	3	
Sub Total		**6**	**Sub Total marks Obtained in Psychomotor(P)**					

B. AFFECTIVE

Sr. No.	Criteria	Allocated Marks	Unacceptable 0%	Poor 25%	Fair 50%	Good 75%	Excellent 100%	Total Obtained
1	Respond to Questions	1	0	0.25	0.5	0.75	1	
2	Lab Report	1	0	0.25	0.5	0.75	1	
3	Assigned Task	2	0	0.5	1	1.5	2	
Sub Total		**4**	**Sub Total marks Obtained in Affective (A)**					

Instructor Name:_____ **Total Marks (P+A):** _____

Instructor Signature:_____ **Obtained Marks (P+A):** _____

STRUCTURING WIDGETS

Before you start developing an app, it's important to create your structure; like when building a house, the foundation is created Structuring widgets in an organized manner improves the code's readability and maintainability. When creating a new Flutter project, the software development kit (SDK) does not automatically create the separate home.dart file, which contains the main presentation page when the app starts. Therefore, to have code separation, you must manually create the pages folder and the home.dart file inside it. The main.dart file contains the main() function that starts the app and calls the Home widget in the home.dart file first.

TRY IT OUT Creating the Dart Files and Widgets

A great way to learn how Flutter works is to start from a blank slate. Delete all the contents of the `main.dart` file. The `main.dart` file has three main sections.

- The `import` package/file
- The `main()` function
- A `class` that extends a `StatelessWidget` widget and returns the app as a widget (like I said before, just about everything is a widget)

Note for the import package that you'll be using Google's Material Design. All the examples in the book import and use Material Design. The Material Design components in a Flutter project are visual, behavioral, and motion widgets. Cupertino can also be used to adhere to Apple's iOS design language that supports iOS-style widgets. You can use both standards in different parts of your app. Right off the bat, Flutter is smart enough to show the native actions in both operating systems without you having to worry about it.

For example, by importing the `cupertino.dart` library, you can mix some of the Cupertino widgets with Material Design. The date and time picker work differently in Android and iOS, and you can specify in the code which widget to show depending on the operating system. However, you'll need to choose up front either Material Design or Cupertino for the entire look and feel of the app. Why? Well, the base of your app needs to be either a `MaterialApp` widget or a `CupertinoApp` widget because this determines the availability of widgets. In step 3 of this exercise, you'll learn how to use the `MaterialApp` widget.

Let's start by adding the code to the `main.dart` file and saving it.

1. Import the package/file. The default import is the `material.dart` library to allow the use of Material Design. (To use the Cupertino iOS-style widgets, you import the `cupertino.dart` library instead of `material.dart`. For the apps in this book, I'll use Material Design.) Then import the `home.dart` page located in the `pages` folder.

```
import 'package:flutter/material.dart';
import 'package:ch4_starter_exercise/pages/home.dart';
```

2. After the two `import` statements, leave a blank line and enter the `main()` function listed next. The `main()` function is the entry point to the app and calls the `MyApp` class.

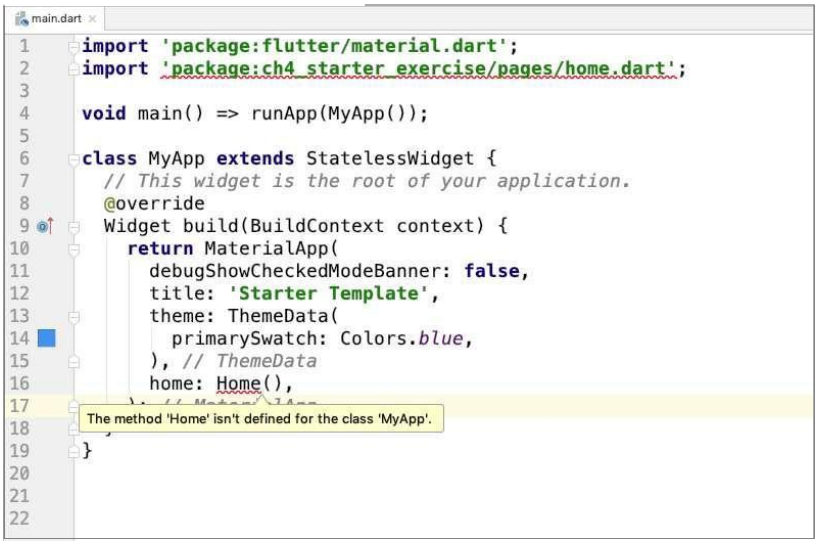
```
void main() => runApp(MyApp());
```

3. Type the `MyApp` class that extends `StatelessWidget`.

The `MyApp` class returns a `MaterialApp` widget declaring `title`, `theme`, and `home` properties. There are many other `MaterialApp` properties available. Notice that the `home` property calls the `Home()` class, which is created later in the `home.dart` file.

```
class MyApp extends StatelessWidget
  // This widget is the root of your application.
  @override
  Widget build(BuildContext context)
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Starter Template'
      theme: ThemeData(
        primarySwatch: Colors.blue,
      )'
      home: Home(),
    );
```

Android Studio shows a red squiggly line under the `import` statement `pages/home.dart` as well as the `Home()` method, which has this error: `The method "Home" isn't defined for the class 'MyApp'`. By hovering your mouse over `pages/home.dart` and `Home()`, you can read each error.



This is normal since you have not created the `home.dart` file containing the home page. This name can be anything you like, but it's always good to have a descriptive name for each page.

4. Create a new Dart file in the `pages` folder. Right-click the `pages` folder, select New ⇨ Dart File, enter **`home.dart`**, and click the OK button to save.
5. Like in step 1, import the `material.dart` package/file. As a reminder, I'll be using Material Design for all the example apps.

```
import 'package:flutter/material.dart';
```

6. Start typing **`st`** and—wow—the autocompletion help opens. As you type the abbreviation for a `StatefulWidget` class, the Android Studio Live Templates automatically fills in the Flutter widget's basic structure. Select the `stful` abbreviation.

7. Now all you need to do is to give the `StatefulWidget` class its name: `Home`. Since it's a **class, the** naming convention is to start the word with an uppercase letter.

```
// home.dart
import 'package:flutter/material.dart';

class Home extends StatefulWidget {
  @override
  _HomeState createState() => _HomeState();


class _HomeState extends State<Home>
  @override
  Widget build(BuildContext context)
    return Container();
```

You're using `StatefulWidget` for the `Home` class because in a real-world application most likely a state would be kept for data. An example of when you would need a state is a `PopupMenuItem`

widget on the AppBar widget showing a selected date used by multiple pages. If the Home class does not need to keep state, then use StatelessWidget.

```
class Home extends StatelessWidget {
  @override
  Widget build(BuildContext context)
    return Container();
```

8. **Replace the** `Container()` **widget with a** `Scaffold` **widget. The** `Scaffold` **widget implements the basic** `MaterialDesign` `visuallayout`,`allowing` `the` `simple` `addition` `of` `AppBar`,`BottomAppBar`, `FloatingActionButton`,`Drawer`,`SnackBar`,`BottomSheet`,`and` `more`.(`If` `this` `were` `a` `CupertinoApp`, `you` `could` `use` `either` `CupertinoPageScaffold` `or` `CupertinoTabScaffold`.)

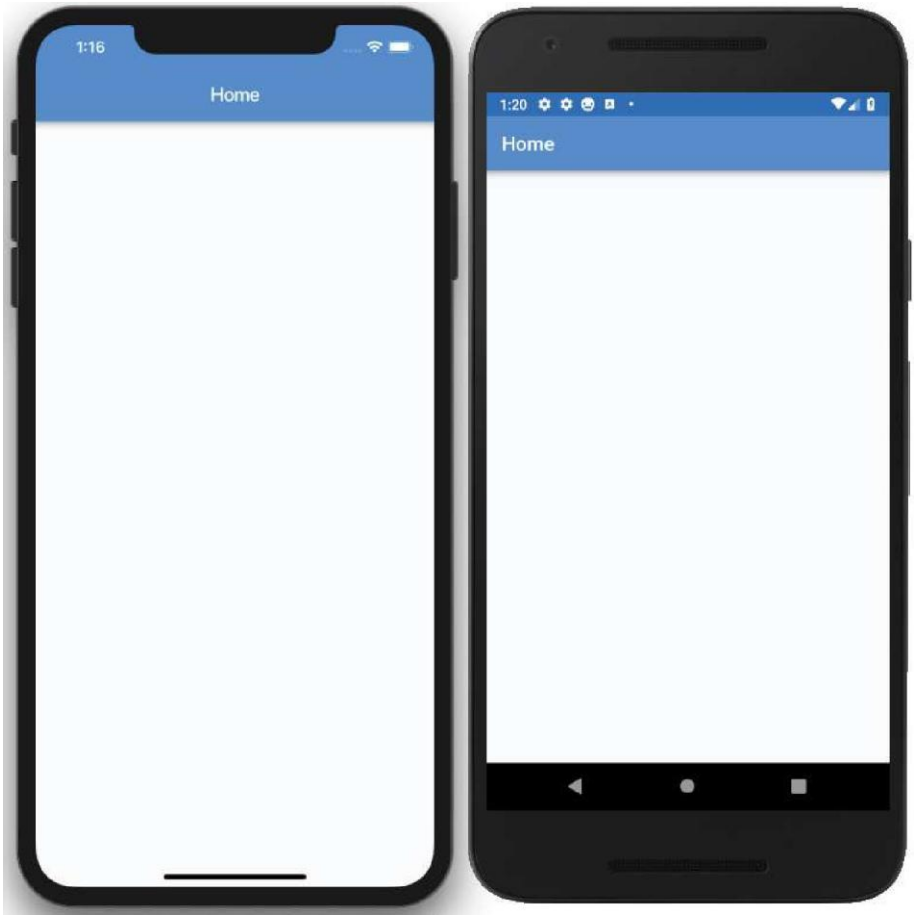
```
class  HomeState extends State<Home>
  @override
  Widget build(BuildContext context)
    return Scaffold(
      appBar: AppBar(
        title: Text{'Home'),
      ) '
      body: Container(),
    );
```

The following is the full source code for both the `main.dart` and `home.dart` files:

```
// main.dart
import 'package:flutter/material.dart';
import 'package:ch4 starter_exercise/pages/home.dart';

void main() => runApp(MyApp());

class MyApp extends StatelessWidget
  // This widget is the root of your application.
  ®override
  Widget build(BuildContext context)
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Starter Template',
      theme: ThemeData(
        primarySwatch: Colors.blue,
      ) '
      home: Home(),
    );



// home.dart
import 'package:flutter/material.dart';

class Home extends StatefulWidget {
  @override
```

```
    HomeState createState() => _HomeState();


class  HomeState extends State<Home>
  @override
  Widget build(BuildContext context)
    return Scaffold(
      appBar:  AppBar(
        title: Text('Home'),
      )'
      body: Container(),
    );
```

Go ahead and run rhe project and see how your app is looking.



Notice that I added the following to `Scaffold` and `AppBar: container` (this can be **a** `Tabcontroller. PageController,` and so on) **and** `FloatingActionButton.`

How It Works

To keep your code readable and maintainable, you structure appropriate widgets in their own classes and Dart files. You structure your starting projects with the `main.dart` file containing the `main()` function that starts the app. The `main()` function calls the `Home` widget in the `home.dart` file. The `Home` widget is the main presentation page shown when the app starts. For example, the `Home` widget might contain a `TabBar` or `BottomNavigationBar` widget.

The `widget tree` is how you create your UI; you position widgets within each other to build simple and complex layouts. Since just about everything in the Flutter framework is a widget, and as you start nesting them, the code can become harder to follow. A good practice is to try to keep the widget tree as shallow as possible. To understand the full effects of a deep tree, you'll look at a `full widget tree` and then refactor it into a `shallow widget tree`, making the code more manageable. You'll learn three ways to create a shallow widget tree by refactoring: with a constant, with a method, and with a widget class.

Introduction to Widget:

Before analyzing the widget tree, let's look at the short list of widgets that you will use for this chapter's example apps. At this point, do not worry about understanding the functionality for each widget; just focus on what happens when you nest widgets and how you can separate them into smaller sections. In Chapter 6, "Using Common Widgets," you'll take a deeper look at using the most common widgets by functionality.

As I mentioned in Chapter 4, "Creating a Starter Project Template," this book uses Material Design for all the examples. The following are the widgets (usable only with Material Design) that you'll use to create the full and shallow widget tree projects for this chapter:

`Scaffold`—Implements the Material Design visual layout, allowing the use of Flutter's Material Components widgets

`AppBar`—Implements the toolbar at the top of the screen

- CircleAvatar-Usually used to show a rounded user profile photo, but you can use it for any image
- Divider-Draws a horizontal line with padding above and below

If the app you are creating is using Cupertino, you can use the following widgets instead. Note that with Cupertino you can use two different scaffolds, a page scaffold or a tab scaffold.

- CupertinoPagescaffold-lmplements the iOS visual layout for a page. It works with CupertinoNavigationBar to provide the use of Flutter's Cupertino iOS-style widgets.
- CupertinoTabScaffold-lmplements the iOS visual layout. This is used to navigate multiple pages, with the tabs at the bottom of the screen allowing you to use Flutter's Cupertino iOS-style widgets.
- CupertinoNavigationBar-lmplements the iOS visual layout toolbar at the top of the screen.

Table 5.1 summarizes, short list of the different widgets to use based on platform.

TABLE 5.1: Material Design vs. Cupertino Widgets

MATERIAL DESIGN	CUPERTINO

- SingleChildScrollview-This adds vertical or horizontal scrolling ability to a single child widget.
- Padding-This adds left, top, right, and bottom padding.
- column-This displays, vertical list of child widgets.
- Row-This displays a horizontal list of child widgets.
- container-This widget can be used as an empty placeholder (invisible) or can specify height, width, color, transform (rotate, move, skew), and many more properties.
- Expanded-This expands and fills the available space for the child widget that belongs to a Column or Row widget.
- Text-The Text widget is a great way to display labels on the screen. It can be configured to be a single line or multiple lines. An optional style argument can be applied to change the color, font, size, and many other properties.

- `Stack`—What a powerful widget! `Stack` lets you stack widgets on top of each other and use a `Positioned (optional)` widget to align each child of the `Stack` for the layout needed. A great example is a shopping cart icon with a small red circle on the upper right to show the number of items to purchase.
- `Positioned`—The `Positioned` widget works with the Stack widget to control child positioning and size. A `Positioned` widget allows you to set the height and width. You can also specify the position location distance from the top, bottom, left, and right sides of the `Stack` widget.

You've learned about each widget that you will implement for the rest of this chapter. You'll now create a full widget tree, and then you'll learn how to refactor it to a shallow widget tree.

BUILDING THE FULL WIDGET TREE

To show how a widget tree can start to expand quickly, you'll use a combination of `Column`, `Row`, `Container`, `CircleAvatar`, `Divider`, `Padding`, and `Text` widgets. You'll take a closer look at these widgets in Chapter 6. The code that you'll write is a simple example, and you can immediately see how the widget tree can grow quickly (Figure 5.1).

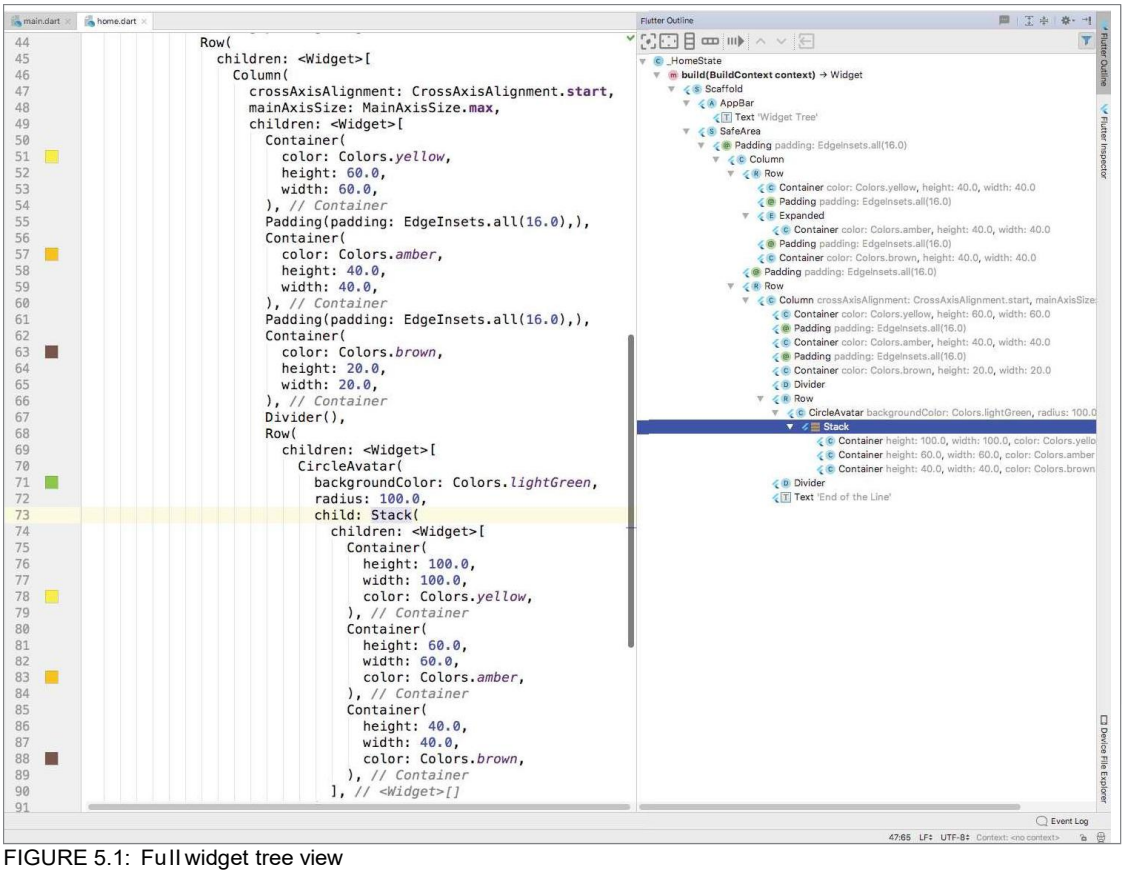


FIGURE 5.1: Full widget tree view

TRY IT OUT Creating the Full Widget Tree

Create a new Flutter project called `ch5_widget_tree`. You can follow the instructions from Chapter 4. For this project, you need co create the `pages` folder only. You can view the full widget tree at the end of the steps.

1. Open the `home.dart:` file.

2. Add to the `Scaffol< body` property a `SafeArea` widget with the `child` property set to a `SingleChildScrollview`. Add a `Padding` widget as a `child` of the `SingleChildScrollView`. Set the `padding` property to `EdgeInsets.all(16.O)`.

```
body: SafeArea(
  child: SingleChildScrollView(
    child: Padding(
      padding: Edgeinsets.all(16.0),
    ) '
  )'
) '
```

3. Add to the `Paddin, child` property a `Column` widget with the `children` property set to a `Row`.

```
body: SafeArea(
  child: SingleChildScrollView(
    child: Padding{
      padding: Edgeinsets.a11(16.0),
      child: Column(
        children: <Widget>[
          Row(
            children: <Widget>[
            J,
          ) '
        ]
      ) '
    ) '
  ) '
) '
```

4. Add to the `Row children` widgets in this order: `Container, Padding, Expanded, Padding, Container,` and `Padding`. You are not done adding widgets; in the next step, you'll add a `Row` widget with **multiple** nested widgets.

```
Row(
  children: <Widget>[
    Container(
      color: Colors.yellow,
      height: 40.0,
      width: 40.0,
    ) '
    Padding(padding: Edgeinsets.all(16.0),),
    Expanded(
      child: Container(
        color: Colors.amber,
        height: 40.0,
```

```
          width: 40.0,
        ),
      ),
      Padding(padding: Edgeinsets.all(16.0),),
      Container(
        color: Colors.brown,
        height: 40.0,
        width: 40.0,
      ),
    ],
  );
```

5. Add a Padding widget to create a space before the next `Row` widget.

```
Padding(padding: Edgeinsets.all(16.0),),
```

6. Add a `Row` widget with the `children` property set to a `Column`. Add to the `Column` `children` a `Container`, `Padding`, `Container`, `Padding`, `Container`, `Divider`, `Row`, `Divider` and `Text`. You are still not done adding widgets, and in the next step, you'll add another `Row` widget with multiple nested widgets.

```
Row(
  children: <Widget>[
    Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      mainAxisSize: MainAxisSize.max,
      children: <Widget>[
        Container(
          color: Colors.yellow,
          height: 60.0,
          width: 60.0,
        ),
        Padding(padding: Edgeinsets.all(16.0),),
        Container(
          color: Colors.amber,
          height: 40.0,
          width: 40.0,
        ),
        Padding(padding: Edgeinsets.all(16.0),),
        Container(
          color: Colors.brown,
          height: 20.0,
          width: 20.0,
        ),
        Divider(),
        Row(
          children: <Widget>[
            // Next step we'll add more widgets
          ],
        ),
        Divider(),
        Text('End of the Line'),
      ],
    ),
  ],
);
```

7. Modify the last `Row` widget (from step 6) and set the `children` property to a `CircleAvatar` with a `child` as a `Stack`. Add to the `Stack children` property three `Container` widgets.

```
Row{
  children: <Widget>[
    CircleAvatar(
      backgroundColor: Colors.lightGreen,
      radius: 100.0,
      child: Stack(
        children: <Widget>[
          Container(
            height: 100.0,
            width: 100.0,
            color: Colors.yellow,
          )'
          Container(
            height: 60.0,
            width: 60.0,
            color: Colors.amber,
          )'
          Container(
            height: 60.0,
            width: 60.0,
            color: Colors.amber,
          )'
        ],
      )'
    )'
  }
)'
```

8. After the `Stack` widget (from step 7), add **a** `Divider` widget and then a `Text` widget with **a** string of `'End of the Line'`.

```
Divider(),
Text('End of the Line'),
```

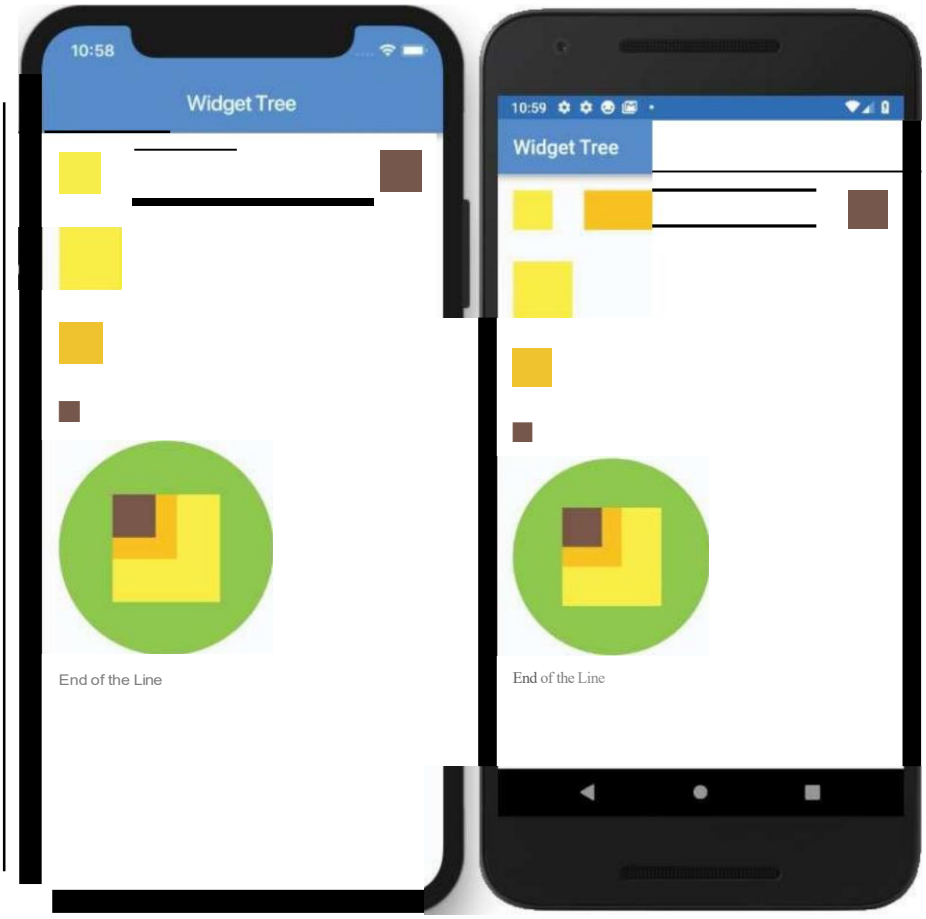
You've added a lot of nested widgets to build a complex layout. The full code is shown next. In a real-world app, this is common. Immediately you'll start co see how the widget tree can grow, making the code unreadable and unmanageable. To keep the example focused on how quickly the widget tree can grow, here you used basic widgets. **In** a production-level app, you would have even more widgets such as text fields to allow the user to enter text.

```
import •package:flutter/mat rial.dart';

class Home extends StatefulWidget {
  @override
   HomeState createState() => _HomeState();


class HomeState extends State<Home>
  @override
  Widget build(BuildContext context)
```

```
return Scaffold(
  appBar:
          AppBar(
    title: Text('Widget       Tree'),
  )'
  body: SafeArea(
    child: SingleChildScrollView(
      child: Padding(
        padding: Edgeinsets.all(16.0),
        child: Column(
          children: <Widget>[
            Row(
              children: <Widget>[
                Container(
                  color: Colors.yellow,
                  height: 40.0,
                  width: 40.0,
                )'
                Padding(padding: Edgeinsets.all{16.0),),
                Expanded(
                  child: Container(
                    color: Colors.amber,
                    height: 40.0,
                    width: 40.0,
                  )'
                )'
                Padding{padding: Edgeinsets.all{16.0),),
                Container(
                  color: Colors.brown,
                  height: 40.0,
                  width: 40.0,
                )'
              ],
            ),
            Padding(padding: Edgeinsets.all{16.0),),
            Row(
              children: <Widget>[
                Column(
                  crossA.xisAlignment: CrossAxisAlignment.start,
                  mainAxisSize: MainAxisSize.max,
                  children: <Widget>[
                    Container(
                      color: Colors.yellow,
                      height: 60.0,
                      width: 60.0,
                    )'
                    Padding{padding:  Edgeinsets.all(16.0),),
                    Container(
                      color: Colors.amber,
                      height: 40.0,
                      width: 40.0,
                    )'
```

```
                    Padding(padding: Edgeinsets.all(16.0),),
                    Container(
                      color: Colors.brown,
                      height: 20.0,
                      width: 20.0,
                    ),
                    Divider(),
                    Row{
                      children: <Widget>[
                        CircleAvatar(
                          backgroundColor: Colors.lightGreen,
                          radius: 100.0,
                          child: Stack(
                            children: <Widget>[
                              Container(
                                height: 100.o,
                                width: 100.0,
                                color: Colors.yellow,
                              ),
                              Containc:r(
                                h;c,ight: 60.0,
                                width: 60.0,
                                color: Colors.amber,
                              ),
                              Container(
                                height: 60.0,
                                width: 60.0,
                                color: Colors.amber,

                              ),
                            ],
                          ),
                        ' '
                      ],
                    ) '
                    Divider(),
                    Text('End of the Line'),
                  ],
                ),
              ],
            ),
          I'
        ),
      ),
    )'
  ),
);
```

The following image shows the page layout resulting from the widget tree.



How It Works

To create a page layout, you nest widgets to create a custom UL The result of adding widgets together is called the *widget tree*. As the number of widgets increases, the widget tree starts to expand quickly and makes the code hard to read and manage.

BUILDING A SHALLOW WIDGET TREE

To make the example code more readable and maintainable, you'll refactor major sections of the code into separate entities. You have multiple refactor options, and the most common technique" are constants, methods, and widget classes.

Refactoring with a Constant

Refactoring with a constant initializes the widget to a `final` variable. his approach allows you to separate widgets into sections, making for better code readability. When widgets are initialized with a constant, they rely on the `BuildContext` object of the parent widget.

What does this mean? Every time the parent widget is redrawn, all the constants will also redraw their widgets, so you can't do any performance optimization. In the next section, you'll take a detailed look at refactoring with a method instead of a constant. The benefits of making the widget tree shallower are similar with both techniques.

The following sample code shows how to use a constant to initialize the `container` variable as `final` with the `Container` widget. You insert the `container` variable in the widget tree where needed.

```
final container= Container(
  color: Colors.yellow,
  height: 40.0,
  width: 40.0,
);
```

Refactoring with a Method

Refactoring with a method returns the widget by calling the method name. The method can l·eturn a value by a general widget (`Widget`) or a specific widget (`container`, `Row`, and others).

The widgets initialized by a method rely on rhe `BuildContext` object of the parent widget. There could be unwanted side effects if these kinds of methods are nested and call other nested methods/functions. Since each situation is different, do nor assume that using methods is nor a good choice. This approach allows you to separate widgets into sections, making for better code readability. However, like when refactoring with a constant, every tin1e the parent widget is redrawn, all the methods will also redraw their widgets. That means the widget tree is not optimizable for performance.

The following sample code shows how to use a method to return a `Container` widget. This first method returns the `Container` widget as a general `Widget`, and the second method returns the `Container` widget as a `Container` widget. Both approaches are acceptable. You insert the `_buildContainer()` method name in the widget tree where needed.

```
II Return by general Widget Name
Widget _buildContainer() (
  return Container(
    color: Colors.yellow,
    height: 40.0,
    width: 40.0,
  );


II Or Return by specific Widget like Container in this case
Container _buildContainer() {
  resgrn Container(
    hei ht,cib?o ·yellow,
```

```
    width: 40.0,
  );
```

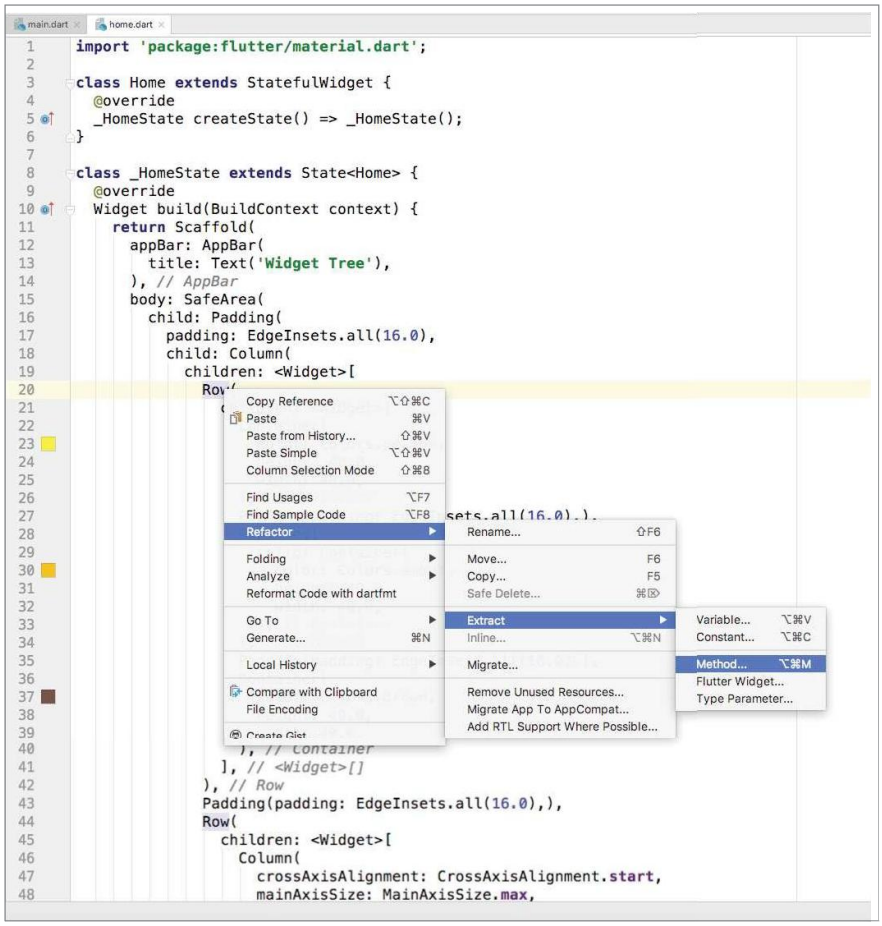
Let's look at an example that refactors by using methods. This approach improves code readability by separating the main parts of the widget tree into separate methods. The same approach could be taken by refactoring with a constant.

What is the benefit of using the method approach? The benefit is pure and simple code readability, but you lose the benefits of Flutter's subtree rebuilding: performance.

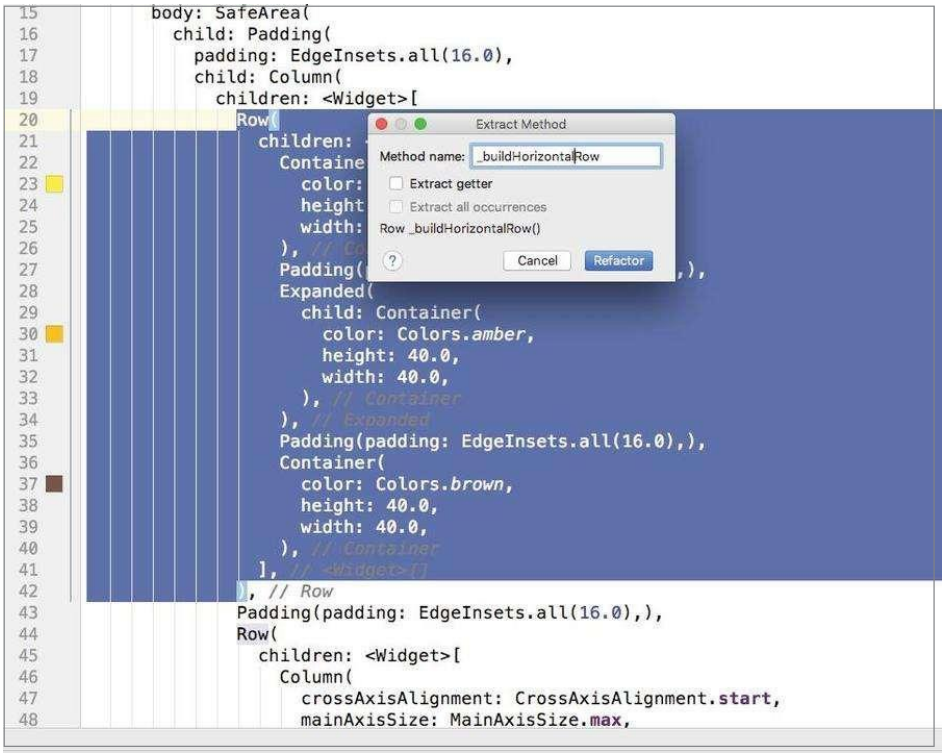
TRY IT OUT Refactoring with a Method to Create a Shallow Widget Tree

To refactor the widgets, use the `method` pattern to flatten the widget tree.

1. Open the `home.dart` file.
2. Place the cursor on the first `Row` widget and right-click.
3. Select Refactor ➪ Extract ➪ Method and click Method.



4. In the Extract Method dialog, enter _buildHorizontalRow for the method name. Notice the underscore before *build*; this lets Dart know that it is a private method. Notice that the entire `Row` widget and the children are highlighted to make it easy to view the affected code.



5. The `Row` widget is replaced with the `_buildHorizontalRow()` method. Scroll to the bottom of the code, and the method and widgets are nicely refactored.

```
Row _buildHorizontalRow() {
  return Row(
    children: <Widget>[
      Container(
        color: Colors.yellow,
        height: 40.0,
        width: 40.0,
      ),
      Padding(padding: EdgeInsets.all(16.0),),
      Expanded(
        child: Container(
          color: Colors.amber,
          height: 40.0,
          width: 40.0,
        ),
      ),
      Padding(padding: EdgeInsets.all(16.0),),
      Container(
        color: Colors.brown,
```

```
        height: 40.0,
        width: 40.0,
      ) '
    ],
  );
```

6. Continue and refactor the other Rows and the Row and `stack` widgets.

The full source code for `home.dart` is shown next. Notice how the widget tree is flattened, making it easier to read. Deciding on how shallow to make the widget tree depend, on each circumstance and your personal preference. For example, say you are working on your code and you start noticing that you are doing a lot of scroJling vertically or horizontally to make changes. This is a good indication that you can refactor portions of the code into separate sections.

```
// home.dart
import 'package,flutter/material.dart';

class Home extends StatefulWidget {
  @override
   HomeState createState() => _HomeState();


class  HomeState extends State<Home>
  @override
  Widget build(BuildContext context)
    return Scaffold(
      appBar: AppBar(
        title: Text('Widget Tree'),
      ) '
      body: SafeArea(
        child: SingleChildScrollView(
          child: Padding(
            padding: Edgeinsets.all(16.0),
            child: Column(
              children: <Widget>[
                _buildHori zontalRow(},
                Padding(padding: Edg Ins ts.al1(16.0),),
                _buildRowAndColumn(1,
              ],
            ),
          ) '
        ) '


  Row _buildHorizontalRow()
    return Row(
      children: <Widget>[
        Container(
          color: Colors.yellow,
          height: 40.0,
          width: 40.0,
        ),
```

```
      Padding(paddin,: Edgeinsets.all(l6.0),),
      Expanded(
        child: Container(
          color: Colors.amber,
          height: 40.0,
          width: 40.0,
        )'
      )'
      Padding[padding: Edgeinsets.all(l6.0),),
      Container(
        color: Colors.brown,
        height: 40.0,
        width: 4O.O,
      )'
    ],
  );


Row _buildRowAndColumn()
  return Row(
    children: <Widget>[
      Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        mainAxisSize: MainAxisSize.max,
        children: <Widget>[
          Container(
            color: Colors.yellow,
            height: 60.0,
            width: 60.0,
          )'
          Padding[padding: Edgeinsets.a11(16.0),),
          Container(
            color: Colors.amber,
            height: 40.0,
            width: 40.0,
          )'
          Padding[padding: Edgeinsets.all(l6.0),),
          Container[
            color: Colors.brown,
            height: 20.0,
            width: 20.0,
          ),
          Divider(),
          _buildRowAndStack(),
          Divider(),           ),
        ,]  Text('End of the Line'
      ),
    ],
  );


Row _buildRowAndStack()
  return Row(
    children: <Widget>[
```

```
        CircleAvatar(
          backgroundColor: Colors.lightGreen,
          radius: 100.0,
          child: Stack (
            children: <Widget>[
              Container(
                height: 100.0,
                width: 100.0,
                color: Colors.yellow,
              ),
              Container(
                height: 60.0,
                width: 60.0,
                color: Colors.amber,
              ),
              Container(
                height: 40.0,
                width: 40.0,
                color: Colors.brown,
              ),
            ],
          ),
        ),
      ],
    );
```

How It Works

Crt:ating a shallow widget trt:e means each widget is separated into its own method by functionality. Keep in mind that how you separate widgets will be different depending on the functionality needed. Separating widgets by 1m:thod improves code readability, but you lose the performance: benefits of Flutter's subtree rebuilding. All the widgets in the method rely on the parent's BuildContext, meaning every time the parent is redrawn, the method is also redrawn.

In this example, you created the _buildHorizontalRow () method co build the horizontal Row widget with child widgets. The _buildRowAndColumn () method is an excellent example of flattening it even more by calling the _buildRowAndStack () method for one of the Column children widgets. Separating _buildRowAndStack () is done to keep the widget tree flat because the _buildRownAndStack () method builds a widget with multiple children widgets.

Refactoring with a Widget Class

Refactoring with a widget class allows you to create the widget by subc1assing the StatelessWidget class. You can create reusable widgets within the current or separate Dart file and initiate them anywhere in the application. Notice that the constructor starts with a const keyword, which allows you to cache and reuse the widget. When calling the constructor to initiate the widget, use the const

keyword. By calling with the const keyword, the widget does not rebuild when other widgets change their state in the tree. If you omit the const keyword, the widget will be called every time the parent widget redraws.

The widget class relies on its own BuildContext, not the parent like the constant and method approaches. BuildContext is responsible for handling the location of a widget in the widget tree. In Chapter 7, "Adding Animation to an App," you'll build an example that refacrors and separates widgets with multiple StatefulWidgets instead of the StatelessWidget class.

What does this mean? Every time the parent widget is redrawn, all the widget classes will not redraw. They are built only once, which is great for performance optimization.

The following sample code shows how to use a widget class to return a Container widget. You insert the const ContainerLeft () widget in the widget tree where needed. Note the use of the const keyword to take advantage of caching.

```
class ContainerLeft extends StatelessWidget {
  const ContainerLeft({
    Key key,
  }) : super{key: key);

  @override
  Widget build, {BuildContext context) {
    return Container(
      color: Colors.yellow,
      height: 40.0,
      width: 40.0,
    );


// Call to initialize the widget and note the const keyword
const ContainerLeft(),
```

Let's look at an exan1ple that refactors by using widget classes (a Flutter widget). This approach improves code readability and performance by separating the main parts of the widget tree into separate widget classes.

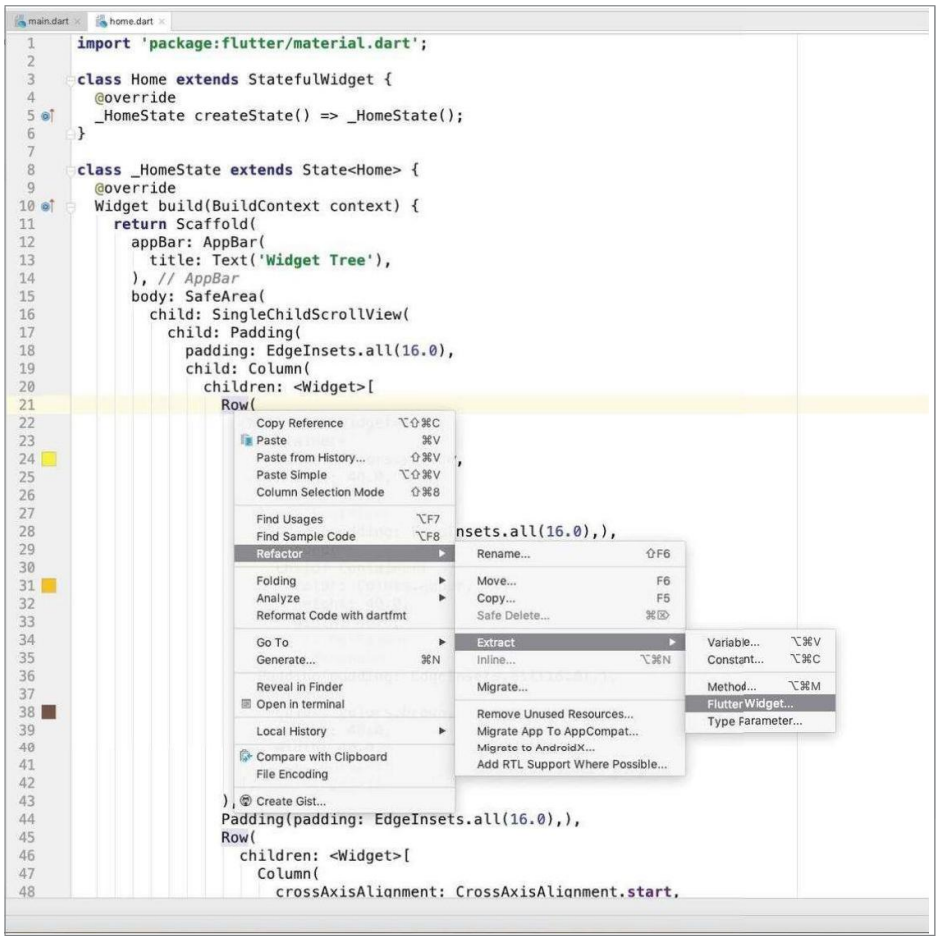
What is the benefit of nsing the widget classes? It's pure and simple performance dLtring screen updates. When calling a widget class, you need to use the const declaration; otherwise, it will he rebuilt every time, without caching. An example of refactoring with a widget class is when you have a UI layout where only specific widgets change state and others stay the same.

TRY IT OUT Refactoring with a Widget Class to Create a Shallow Widget Tree

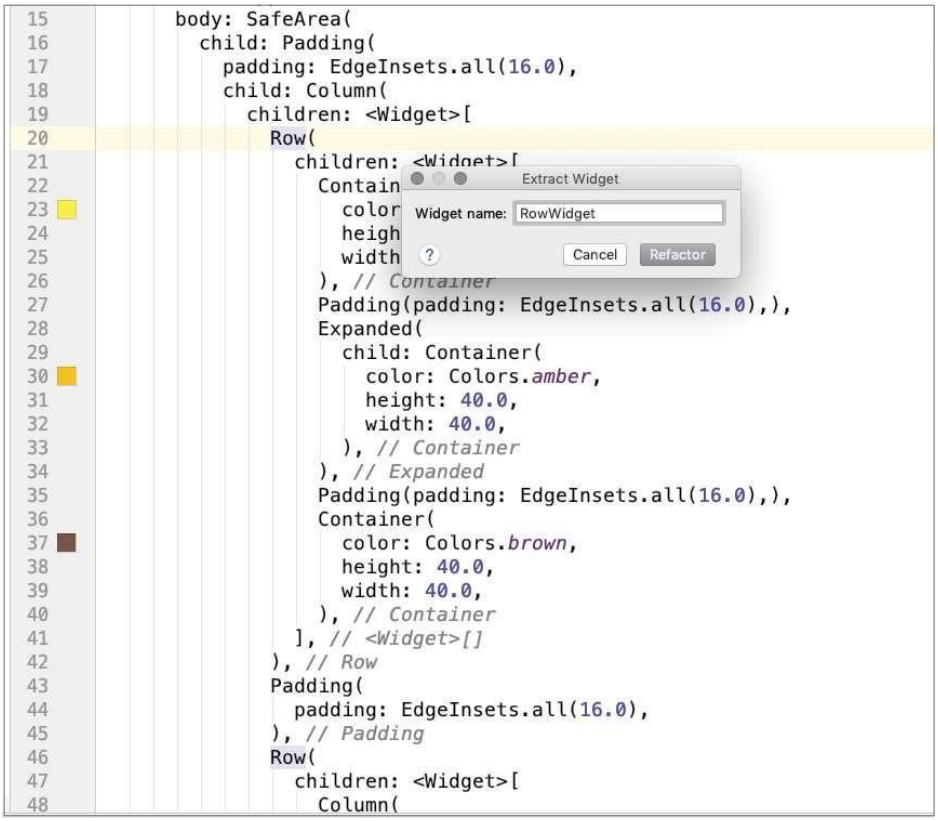
Create a new Flutter project called ch5_widget_tree_performance. You can follow the instructions from Chapter 4. For this project, you need to create the pages folder only. To keep this example simplt:,

you'll create the widget classes in the `home.dart` file, but in Chapter 7 you'll learn how to separate them into separate files.

1. Open the `home.dart` file. Copy the original full widget tree in `home.dart` (from the "Building the Full Widget Tree" section of this chapter) to this project's `home.dart` file.
2. Place the cursor on the first `Row` widget and right-click.
3. Select Refactor ➪ Extract ➪ Flutter Widget.



4. In the Extract Widget dialog, enter **RowWidget** for the widget name.




```
body: SafeArea(
  child: Padding(
    padding: EdgeInsets.all(16.0),
    child: Column(
      children: <Widget>[
        Row(
          children: <Widget>[
            Contain
              color
              heigh
              width
            ), // Container
            Padding(padding: EdgeInsets.all(16.0),),
            Expanded(
              child: Container(
                color: Colors.amber,
                height: 40.0,
                width: 40.0,
              ), // Container
            ), // Expanded
            Padding(padding: EdgeInsets.all(16.0),),
            Container(
              color: Colors.brown,
              height: 40.0,
              width: 40.0,
            ), // Container
          ], // <Widget>[]
        ), // Row
        Padding(
          padding: EdgeInsets.all(16.0),
        ), // Padding
        Row(
          children: <Widget>[
            Column(
```

5. The `Row` widget is replaced with the `RowWidget()` widget class. Since the `Row` widget will not change state, add the `const` keyword before calling the `RowWidget()` class. Scroll to the bottom of the code, and the widgets are nicely refactored into the `RowWidget` (`StatelessWidget`) class.

```
class RowWidget extends StatelessWidget {
  const RowWidget({
    Key key,
  }) : super(key: key);

  @override
  Widget build(BuildContext context) {
    print('RowWidget');

    return Row(
      children: <Widget>[
        Container(
          color: Colors.yellow,
          height: 40.0,
          width: 40.0,
        ),
        Padding(
          padding: EdgeInsets.all(16.0),
        ),
```

```
        Expanded(
          child, Container(
            color: Colors.amber,
            height: 40.0,
            width, 40.0,
          )'
        )'
        Padding(
          padding: Edgeinsets.a11{16.0),
        )'
        Container(
          color: Colors.brown,
          height: 40 .0,
          width: 4o.o,
        )'
      ]'
    );
```

6. Continue and refactor the other `Rows` (RowAndColumnWidget class) and the `Row` and `stack` (RowAndStackWidget class) widgets.

The full source code for `home.dart` is listed next. Notice how the widget tree is flattened, making it easier to read. Deciding on how shallow to make the widget tree depends on each circumstance and your personal preference.

```
// home.dart
import 'package:flutter/material.dart';

class Home extends StatefulWidget {
  @override
   HomeState createState() => _HomeState();


class  HomeState extends State<Home>
  @override
  Widget build(BuildContext context)
    return Scaffold(
      appBar: AppBar(
        title: Text('Widget Tree'),
      )'
      body: SafeArea(
        child: SingleChildScrollView(
          child: Padding(
            padding: Edgeinsets.all(l6.0),
            child: Column(
              children: <Widget>[
                const RowWidget(),
                Padding(
                  paddini: Edgeinsets.all{16.0),
                66nst
                      RowAndColumnWidget(),
              ],
            )'
```

```
        ))'
    ); '
```

```
class RowWidget extends StatelessWidget {
  const RowWidget({
    Key key,
  )) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return Row(
      children: <Widget>[
        Container(
          color: Colors.yellow,
          height: 40. 0,
          width: 40.0,
        )'
        Padding(
          padding: Edgeinsets.all(l6.0),
        )'
        Expanded(
          child: Container(
            color: Colors.amber,
            height: 40.0,
            width: 40.0,
          )'
        )'
        Padding(
          padding: Edgeinsets.all(l6.0),
        )'
        Container(
          color: Colors.brown,
          height: 40.0,
          width: 40.0,
        )'
      ),
    );
```

```
class RowAndColumnWidget extends StatelessWidget {
  const RowAndColumnWidget({
    Key key,
  }) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return Row(
      children: <Widget>[
        Column(
          crossAxisAlignment: CrossAxisAlignment.start,
```

```
          mainAxisSize: MainAxisSize.max,
          children: <Widget>[
            Container(
              color: Colors.yellow,
              height: 60.0,
              width: 60.0,
            )'
            Padding|
              paddin(  EdgeInsets.all(16.0),
            )'
            Container(
              color: Colors.amber,
              height: 40.0,
              width: 40.0,
            )'
            Padding(
              paddin   Edgeinsets.all(16.0),
            )'
            Container(
              color: Colors.brown,
              height: 20.0,
              width: 20.0,
            )'
            Divider(),
            const RowAndStackWidget(),
            Divider(),
            Text('End of the Line. Date: ${DateTime.now()}'),
          ],
        )'
      ],
    );
```

```
class RowAndStackWidget extends Statelesswidget {
  const RowAndStackWidget({
    Key key,
  }) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return Row(
      children: <Widget>[
        CircleAvatar(
          backgroundColor: Colors.lightGreen,
          radius: 100.0,
          child: Stack(
            children: <Widget>[
              Container(
                height: 100.0,
                width: 100.0,
                color: Colors.yellow,
              ),height:
              Container(
                        60.0,
```

```
            width: 60.0,
            color: Colors.amber,
          ),
          Container(
            height: 40.0,
            width: 40.0,
            color: Colors.brown,
          ),
        ],
      ),
    ),
  ),
);
```

Lab Task

Profile & Posts App

You will create a simple profile and posts app that displays a user profile with an avatar, name, and bio at the top, followed by a list of posts. The structure of the App will be as given bellow.

Widget	Usage
MaterialApp	Wraps the entire app
Scaffold	Provides the basic structure with an AppBar and body
AppBar	Displays the top navigation title
Column	Organizes profile and posts vertically
Row	Aligns the profile picture and user details horizontally
CircleAvatar	Displays the user's profile picture
Text	Shows the user's name, bio, and post content
Stack	Overlays the Floating Action Button
SingleChildScrollView	Makes the list scrollable
Padding	Adds spacing around widgets
Container	Styles each post with padding and shadow
Expanded	Ensures the text takes the available space in the row
Divider	Creates a separator between profile and posts

Main.dart:

```dart
import 'package:flutter/material.dart';
import 'widgets/user_profile.dart';

void main() => runApp(MaterialApp(
  home: Scaffold(
    appBar: AppBar(title: Text('Test - UserProfile')),
    body: Padding(
      padding: EdgeInsets.all(16.0),
      child: UserProfile(),
    ), Padding
  ), Scaffold
)); MaterialApp
```

user_profile.dart:

```dart
import 'package:flutter/material.dart';

class UserProfile extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Row(
      children: <Widget>[
        Stack(
          children: <Widget>[
            CircleAvatar(
              radius: 36,
              backgroundColor: Color(0xFFC0392B),
              child: Text(
                'AH',
                style: TextStyle(
                  color: Colors.white,
                  fontSize: 22,
                  fontWeight: FontWeight.bold,
                ), TextStyle
              ), Text
            ), CircleAvatar
            Container(
```

```
      margin: EdgeInsets.only(top: 52, left: 52),
      width: 14,
      height: 14,
      decoration: BoxDecoration(
        color: Colors.green,
        shape: BoxShape.circle,
      ), BoxDecoration
    ), Container
  ], <Widget>[]
), Stack

// ✅ WIDGET: Padding
// Adds horizontal spacing between the avatar and the text column.
Padding(padding: EdgeInsets.all(12.0)),

// ✅ WIDGET: Expanded
// Makes the text column fill all remaining horizontal space in the Row.
Expanded(
  child: Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: <Widget>[
          Text(
            'Ahmad',
            style: TextStyle(
              fontSize: 20,
              fontWeight: FontWeight.bold,
              color: Color(0xFF2C3E50),
            ), TextStyle
          ), Text

          Text(
            'Patient ID: HV-2024-001',
            style: TextStyle(fontSize: 13, color: Colors.grey),
          ), Text

          Text(
            'Last Report: 12 May 2025',
            style: TextStyle(fontSize: 13, color: Colors.grey),
          ), Text
        ], <Widget>[]
      ), Column
    ), Expanded
  ], <Widget>[]
); Row
  }
}
```

Test - UserProfile

Ahmad

Patient ID: HV-2024-001

Last Report: 12 May 2025

AH

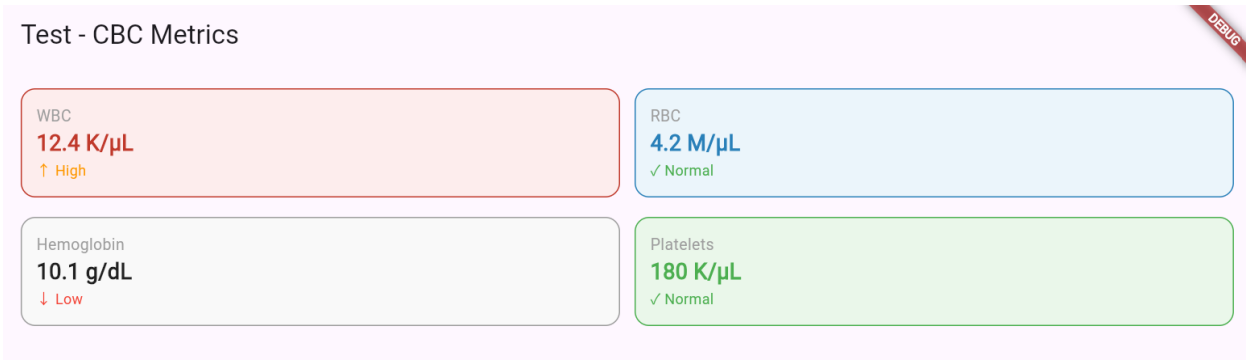
Main.dart:

```dart
import 'package:flutter/material.dart';
import 'widgets/cbc_metrics.dart';

void main() => runApp(MaterialApp(
  home: Scaffold(
    appBar: AppBar(title: Text('Test - CBC Metrics')),
    body: Padding(
      padding: EdgeInsets.all(16.0),
      child: CbcMetrics(),
    ), Padding
  ), Scaffold
)); MaterialApp
```

cbc_metrics.dart:

```dart
import 'package:flutter/material.dart';
class CbcMetrics extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Column(
      children: <Widget>[
        Row(
          children: <Widget>[
            Expanded(
              child: _buildMetricCard(
                label: 'WBC',
                value: '12.4 K/µL',
                status: '↑ High',
                borderColor: Color(0xFFC0392B),
                bgColor: Color(0xFFFDEDED),
                valueColor: Color(0xFFC0392B),
                statusColor: Colors.orange,
              ),
            ), Expanded
            Padding(padding: EdgeInsets.all(6.0)),
            Expanded(
              child: _buildMetricCard(
                label: 'RBC',
                value: '4.2 M/µL',
                status: '✓ Normal',
                borderColor: Color(0xFF2980B9),
                bgColor: Color(0xFFEBF5FB),
                valueColor: Color(0xFF2980B9),
                statusColor: Colors.green,
              ),
            ), Expanded
          ], <Widget>[]
        ), Row

        Padding(padding: EdgeInsets.all(8.0)),
        Row(
          children: <Widget>[
            Expanded(
              child: _buildMetricCard(
                label: 'Hemoglobin',
                value: '10.1 g/dL',
                status: '↓ Low',
                borderColor: Colors.grey,
```

```
                bgColor: Color(0xFFF9F9F9),
                valueColor: Colors.black87,
                statusColor: Colors.red,
              ),
            ), Expanded

            Padding(padding: EdgeInsets.all(6.0)),

            Expanded(
              child: _buildMetricCard(
                label: 'Platelets',
                value: '180 K/µL',
                status: '✓ Normal',
                borderColor: Colors.green,
                bgColor: Color(0xFFEAF7EA),
                valueColor: Colors.green,
                statusColor: Colors.green,
              ),
            ), Expanded
          ], <Widget>[]
        ), Row
      ], <Widget>[]
    ); Column
  }
  Widget _buildMetricCard({
    required String label,
    required String value,
    required String status,
    required Color borderColor,
    required Color bgColor,
    required Color valueColor,
    required Color statusColor,
  }) {
    return Container(
      height: 90,
      decoration: BoxDecoration(
        color: bgColor,
        borderRadius: BorderRadius.circular(12),
        border: Border.all(color: borderColor, width: 1),
      ), BoxDecoration
      child: Padding(
        padding: EdgeInsets.all(12.0),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: <Widget>[

            // ✅ WIDGET: Text — metric label (e.g. WBC, RBC)
            Text(label, style: TextStyle(fontSize: 13, color: Colors.grey)),

            // ✅ WIDGET: Text — metric value (e.g. 12.4 K/µL)
            Text(
              value,
              style: TextStyle(
                fontSize: 18,
                fontWeight: FontWeight.bold,
                color: valueColor,
              ), TextStyle
            ), Text
            Text(status, style: TextStyle(fontSize: 12, color: statusColor)),
          ], <Widget>[]
        ), Column
      ), Padding
    ); Container
```



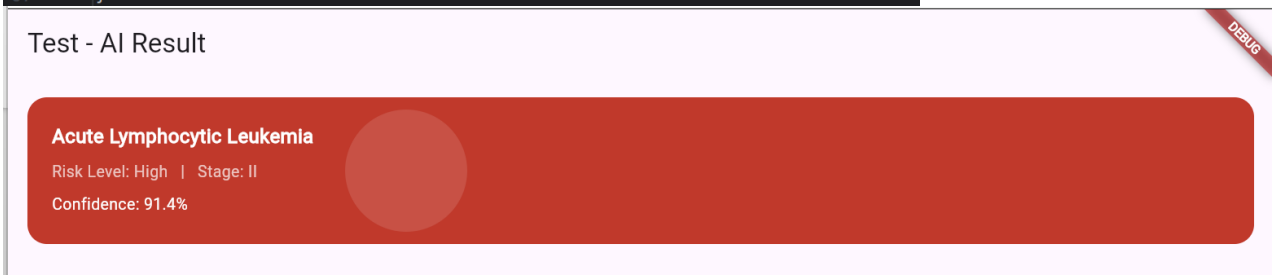
Main.dart:

```dart
import 'package:flutter/material.dart';
import 'widgets/ai_result_card.dart';

void main() => runApp(MaterialApp(
  home: Scaffold(
    appBar: AppBar(title: Text('Test - AI Result')),
    body: Padding(
      padding: EdgeInsets.all(16.0),
      child: AiResultCard(),
    ), Padding
  ), Scaffold
)); MaterialApp
```

ai_result_card.dart :

```dart
import 'package:flutter/material.dart';

class AiResultCard extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Stack(
      children: <Widget>[
        Container(
          width: double.infinity,
          height: 120,
          decoration: BoxDecoration(
            color: Color(0xFFC0392B),
            borderRadius: BorderRadius.circular(16),
          ), BoxDecoration
        ), Container
        Container(
          width: 100,
          height: 100,
          margin: EdgeInsets.only(left: 260, top: 10),
          decoration: BoxDecoration(
            color: Colors.white12,
```

```
            shape: BoxShape.circle,
          ), BoxDecoration
        ), Container
        Padding(
          padding: EdgeInsets.all(20.0),

          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: <Widget>[
              Text(
                'Acute Lymphocytic Leukemia',
                style: TextStyle(
                  color: Colors.white,
                  fontSize: 16,
                  fontWeight: FontWeight.bold,
                ), TextStyle
              ), Text

              Padding(padding: EdgeInsets.all(4.0)),
              Text(
                'Risk Level: High   |   Stage: II',
                  style: TextStyle(color: Colors.white70, fontSize: 13),
                ), Text

                Padding(padding: EdgeInsets.all(4.0)),
                Text(
                  'Confidence: 91.4%',
                  style: TextStyle(color: Colors.white, fontSize: 13),
                ), Text
              ], <Widget>[]
            ), Column
          ), Padding
        ], <Widget>[]
      ); Stack
    }
}
```



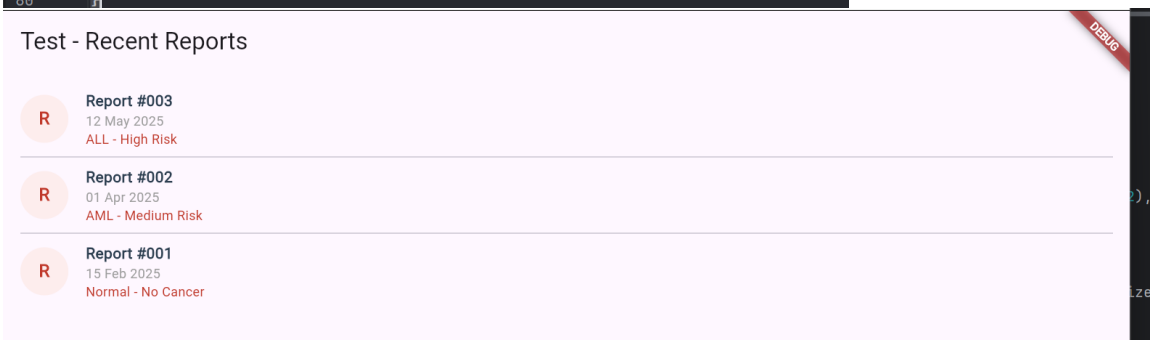
Main.dart:

```
import 'package:flutter/material.dart';
import 'widgets/recent_reports.dart';

void main() => runApp(MaterialApp(
  home: Scaffold(
    appBar: AppBar(title: Text('Test - Recent Reports')),
    body: Padding(
      padding: EdgeInsets.all(16.0),
      child: RecentReports(),
    ), Padding
  ), Scaffold
)); MaterialApp
```

recent_reports.dart:

```dart
import 'package:flutter/material.dart';

class RecentReports extends StatelessWidget {
  final List<Map<String, String>> reports = [
    {
      'title': 'Report #003',
      'date': '12 May 2025',
      'result': 'ALL - High Risk',
    },
    {
      'title': 'Report #002',
      'date': '01 Apr 2025',
      'result': 'AML - Medium Risk',
    },
    {
      'title': 'Report #001',
      'date': '15 Feb 2025',
      'result': 'Normal - No Cancer',
    },
  ];

  @override
  Widget build(BuildContext context) {
    return Column(
      children: <Widget>[

        _buildReportRow(reports[0]),
        Divider(),

        _buildReportRow(reports[1]),

        Divider(),

        _buildReportRow(reports[2]),
      ], <Widget>[]
    ); Column
  }
  Widget _buildReportRow(Map<String, String> report) {
    return Row(
      children: <Widget>[
        CircleAvatar(
          radius: 22,
          backgroundColor: Color(0xFFFDEDED),
          child: Text(
            'R',
            style: TextStyle(
              color: Color(0xFFC0392B),
              fontWeight: FontWeight.bold,
              fontSize: 16,
            ), TextStyle
          ), Text
        ), CircleAvatar
        Padding(padding: EdgeInsets.all(8.0)),
        Expanded(
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: <Widget>[
              Text(
                report['title']!,
                style: TextStyle(
                  fontWeight: FontWeight.bold,
                  fontSize: 14,
                  color: Color(0xFF2C3E50),
                ), TextStyle
```

```
            ), Text
            Text(
              report['date']!,
              style: TextStyle(color: Colors.grey, fontSize: 12),
            ), Text
            Text(
              report['result']!,
              style: TextStyle(color: Color(0xFFC0392B), fontSize: 12),
            ), Text
          ], <Widget>[]
        ), Column
      ), Expanded
    ], <Widget>[]
  ); Row
 }
}
```



Full screen